sson 1: The basics of C++

This tutorial is for everyone; if you've never programmed before, or if you have extensive experience programming in other languages and want to expand into C++. It is for everyone who wants the feeling of accomplishment from a working program.

C++ is a programming language of many different dialects, just as each spoken language has many different dialects. In C++, dialects are not because the speakers live in the North or South; it is because there are several different compilers. There are several common compilers: Borland C++, Microsoft C++, GNU C++, etc.. There are also many front-end environments for the different compilers, the most common is Dev-C++ around GNU's G++ compiler. Some are free, others are not. Please see the compiler listing on this site for information on how to get one and set it up.

Each of these compilers is slightly different. Each one should support the ANSI/ISO standard C++ functions, but each compiler will also have nonstandard functions (these functions are similar to slang spoken in different parts of a country. Sometimes the use of nonstandard functions will cause problems when you attempt to compile source code (the actual C++ written by a programmer and saved as a text file) with a different compiler. These tutorials use ANSI/ISO standard C++ and should not suffer from this problem with sufficiently modern compilers.

If you don't have a compiler, I strongly suggest that you get a compiler. A simple compiler is sufficient for out use, but you should get one in order to get the most from these tutorials.

C++ is a different breed of programming language. A C++ program begins with a function, a collection of commands that do something. The function that begins a C++ program is called main. From main, we can also call other functions whether they be written by us or by others. To access a standard function that comes with the compiler, you include a header with the #include directive. What this does is take everything in the header and paste it into your program. Let's look at a working program:

```
#include <iostream>

using namespace std;

int main()
{
  cout<<"HEY, you, I'm alive! Oh, and Hello World!\n";
  cin.get();
}
```

Let's look at the elements of the program. The #include is a preprocessor directive that tells the compiler to put code from the header called iostream into our program. By including header files, you an gain access to many different functions. For example, the cout function requires iostream.

The next imporant line is int main(). This line tells the compiler that there is a function named main, and that the function returns an integer, hence int. The braces ({ and }) signal the beginning and end of functions and other code blocks. If you have programmed in Pascal, you will know them as BEGIN and END.

The next line of the program may seem strange. If you have programmed in another language, you might expect that print would be the function used to display text. In C++, however, the cout function is used to display text. It uses the << symbols, known as insertion operators. The quotes tell the compiler

that you want to output the literal string as-is. The \n is actually one character that stands for a newline. It moves the cursor on your screen to the next line. The semicolon is added onto the end of all function calls in C++. You will see later that the semicolon is used to end most commands in C++.

The next command is cin.get(). This is another function call that expects the user to hit the return key. Many compiler environments will open a new console window, run the program, and then close the window. This command keeps that window from closing because the program is not done yet. It gives you time to see the program run.

Upon reaching the end of main, the closing brace, our program will return the value of 0 (and integer, hence why we told main to return an int) to the operating system. This return value is important as it can be used to tell the OS whether our program succeeded or not. A return value of 0 means success and is returned automatically (but only for main, other functions require you to manually return a value), but if we wanted to return something else, such as 1, we would have to do it with a return statement:

```
#include <iostream>

using namespace std;

int main()
{
  cout<<"HEY, you, I'm alive! Oh, and Hello World!\n";
  cin.get();

  return 1;
}
```

The final brace closes off the function. You should try compiling this program and running it. You can cut and paste the code into a file, save it as a .cpp (or whatever extension your compiler requires) file. If you are using a command-line compiler, such as Borland C++ 5.5, you should read the compiler instructions for information on how to compile. Otherwise compiling and running should be as simple as clicking a button with your mouse.

Comments are critical for all but the most trivial programs. When you tell the compiler a section of text is a comment, it will ignore it when running the code, allowing you to use any text you want to describe the real code. To create a comment use either //, which tells the compiler that the rest of the line is a comment, or /* and then */ to block off everything between as a comment. Certain compiler environments will change the color of a commented area, but some will not. Be certain not to accidentally comment out code (that is, to tell the compiler part of your code is a comment) you need for the program. When you are learning to program, it is useful to be able to comment out sections of code in order to see how the output is affected.

So far you should be able to write a simple program to display information typed in by you, the programmer. It is also possible for your program to accept input. The function you use is known as cin, and is followed by the insertion operator >>.

Of course, before you try to receive input, you must have a place to store that input. In programming, input and data are stored in variables. There are several different types of variables; when you tell the compiler you are declaring a variable, you must include the data type along with the name of the variable. Several basic types include char, int, and float.

A variable of type char stores a single character, variables of type int store integers (numbers without

decimal places), and variables of type float store numbers with decimal places. Each of these variable types - char, int, and float - is each a keyword that you use when you declare a variable. To declare a variable you use the syntax type <name>. It is permissible to declare multiple variables of the same type on the same line; each one should be separated by a comma. The declaration of a variable or set of variables should be followed by a semicolon (Note that this is the same procedure used when you call a function). If you attempt to use an undefined variable, your program will not run, and you will receive an error message informing you that you have made a mistake.

Here are some variable declaration examples:

```
int x;
int a, b, c, d;
char letter;
float the_float;
```

While you can have multiple variables of the same type, you cannot have multiple variables with the same name. Moreover, you cannot have variables and functions with the same name.

```
#include <iostream>

using namespace std;

int main()
{
  int thisisanumber;

  cout<<"Please enter a number: ";
  cin>> thisisanumber;
  cin.ignore();
  cout<<"You entered: "<< thisisanumber <<"\n";
  cin.get();
}
```

Let’s break apart this program and examine it line by line. The keyword int declares thisisanumber to be an integer. The function cin>> reads a value into thisisanumber; the user must press enter before the number is read by the program. cin.ignore() is another function that reads and discards a character. Remember that when you type intput into a program, it takes the enter key too. We don’t need this, so we throw it away. Keep in mind that the variable was declared an integer; if the user attempts to type in a decimal number, it will be truncated (that is, the decimal component of the number will be ignored). Try typing in a sequence of characters or a decimal number when you run the example program; the response will vary from input to input, but in no case is it particularly pretty. Notice that when printing out a variable quotation marks are not used. Were there quotation marks, the output would be "You Entered: thisisanumber." The lack of quotation marks informs the compiler that there is a variable, and therefore that the program should check the value of the variable in order to replace the variable name with the variable when executing the output function. Do not be confused by the inclusion of two separate insertion operators on one line. Including multiple insertion operators on one line is acceptable as long as each insertion operator outputs a different piece of information; you must separate string literals (strings enclosed in quotation marks) and variables by giving each its own insertion operators (<<). Trying to put two variables together with only one << will give you an error message, do not try it. Do not forget to end functions and declarations with a semicolon. If you forget the semicolon, the compiler will give you an error message when you attempt to compile the program. Variables are uninteresting without the ability to modify them. Several operators used with variables include the

following: *, -, +, /, =, ==, >, <. The * multiplies, the - subtracts, and the + adds. It is of course important to realize that to modify the value of a variable inside the program it is rather important to use the equal sign. In some languages, the equal sign compares the value of the left and right values, but in C++ == is used for that task. The equal sign is still extremely useful. It sets the left input to the equal sign, which must be one, and only one, variable equal to the value on the right side of the equal sign. The operators that perform mathematical functions should be used on the right side of an equal sign in order to assign the result to a variable on the left side.

Here are a few examples:

```
a = 4 * 6; // (Note use of comments and of semicolon) a is 24
a = a + 5; // a equals the original value of a with five added to it
a == 5     // Does NOT assign five to a. Rather, it checks to see if a equals 5.
```

The other form of equal, ==, is not a way to assign a value to a variable. Rather, it checks to see if the variables are equal. It is useful in other areas of C++; for example, you will often use == in such constructions as conditional statements and loops. You can probably guess how < and > function. They are greater than and less than operators.

For example:

```
a < 5  // Checks to see if a is less than five
a > 5  // Checks to see if a is greater than five
a == 5 // Checks to see if a equals five, for good measure
```

Home Tutorials
Quiz yourself Next: If Statements